

CHALLENGE #3: ORDER ORDER Write-up.

ORDER ORDER

Hard

Web Exploitation

picoCTF 2026

AUTHOR: DARKRAICG492

Description

Can you try to get the flag from our website. I've prepared my queries everywhere! I think!

Additional details will be available after launching your challenge instance.

This challenge launches an instance on demand.

Its current status is:

NOT_RUNNING

Launch Instance

Hints

1

What does order in SQL Injection mean?

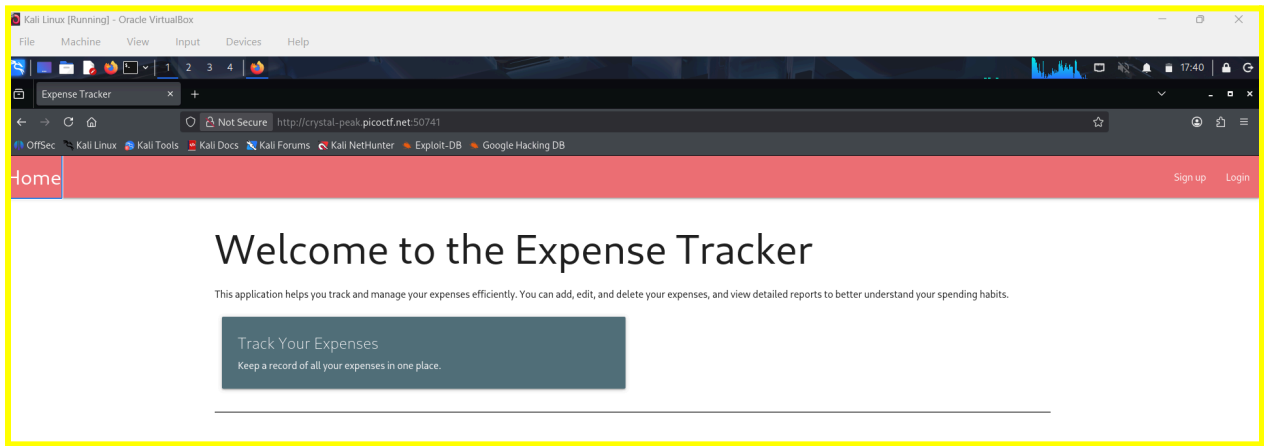
693 users solved

91% Liked

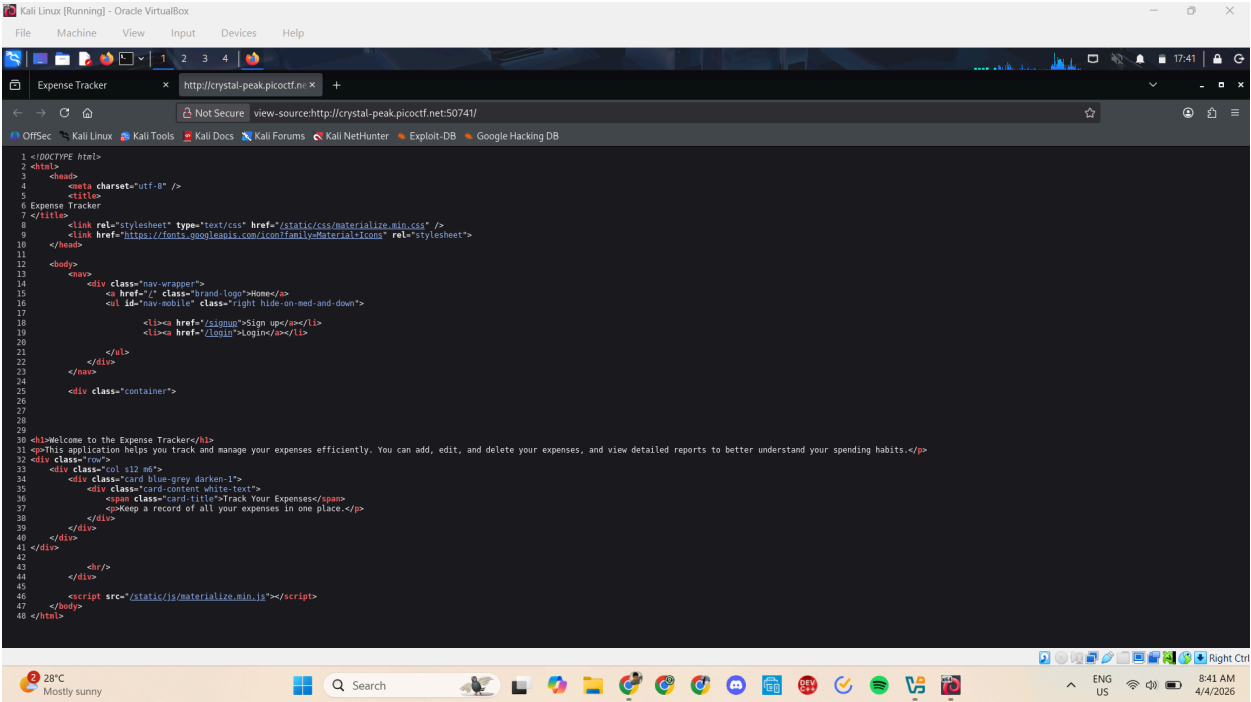
picoCTF{FLAG}

Submit Flag

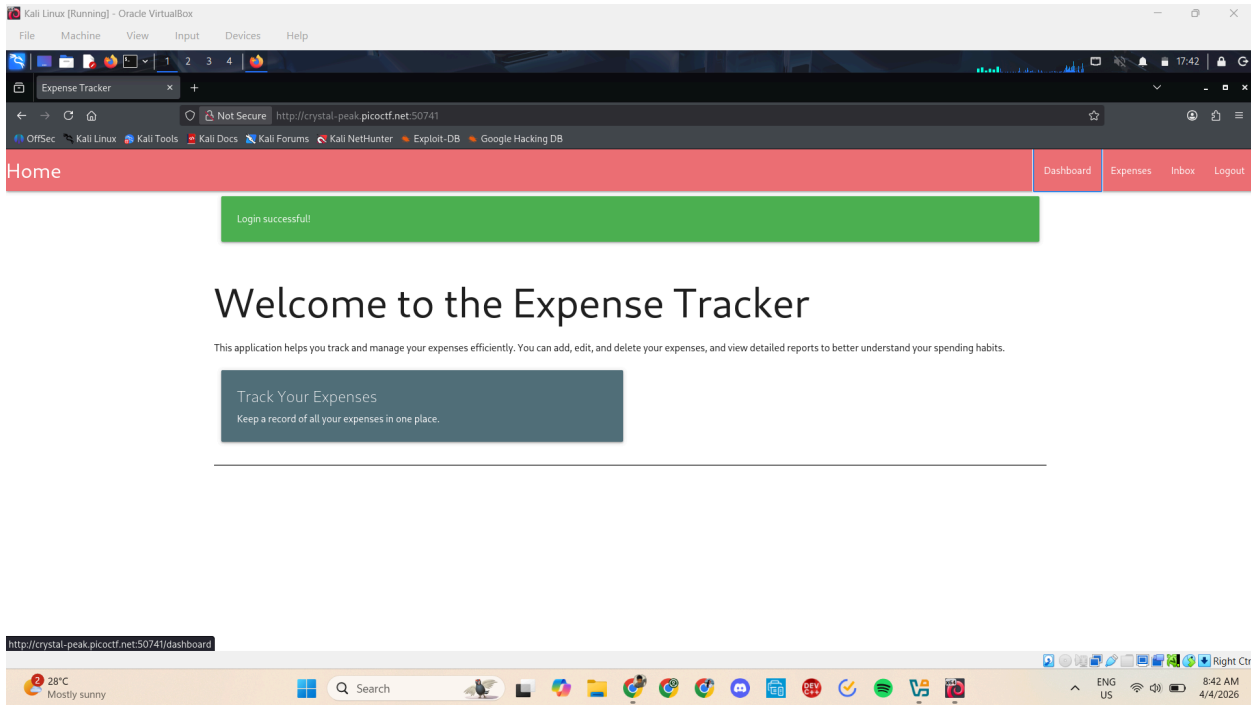
Solving Process:



The challenge began by opening the provided website link, which led to an **Expense Tracker web application**. The application allowed users to register, log in, add expenses, generate reports, and download expense summaries. At first glance, the website appeared to function normally, and no obvious vulnerabilities were visible from the page source. Therefore, the next step was to interact with the application as a normal user to understand how it worked.

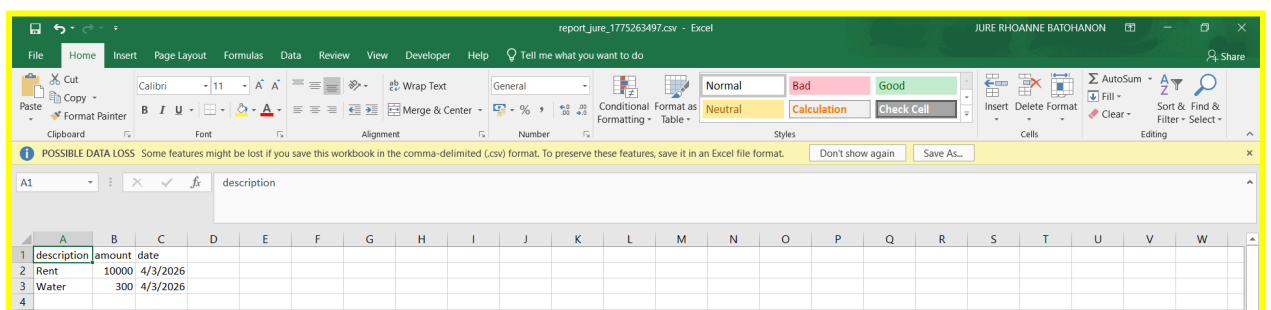
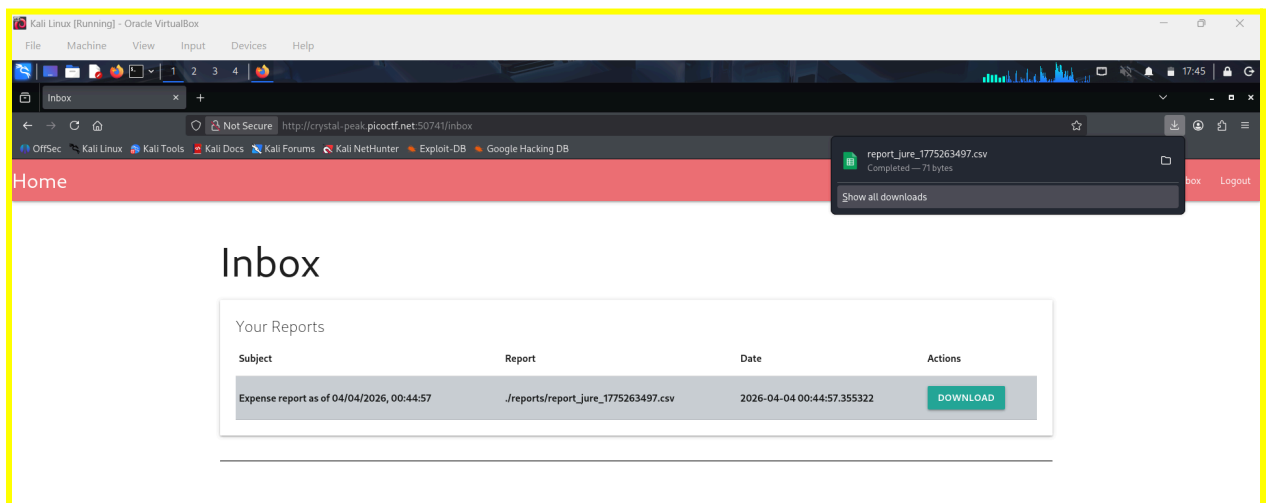
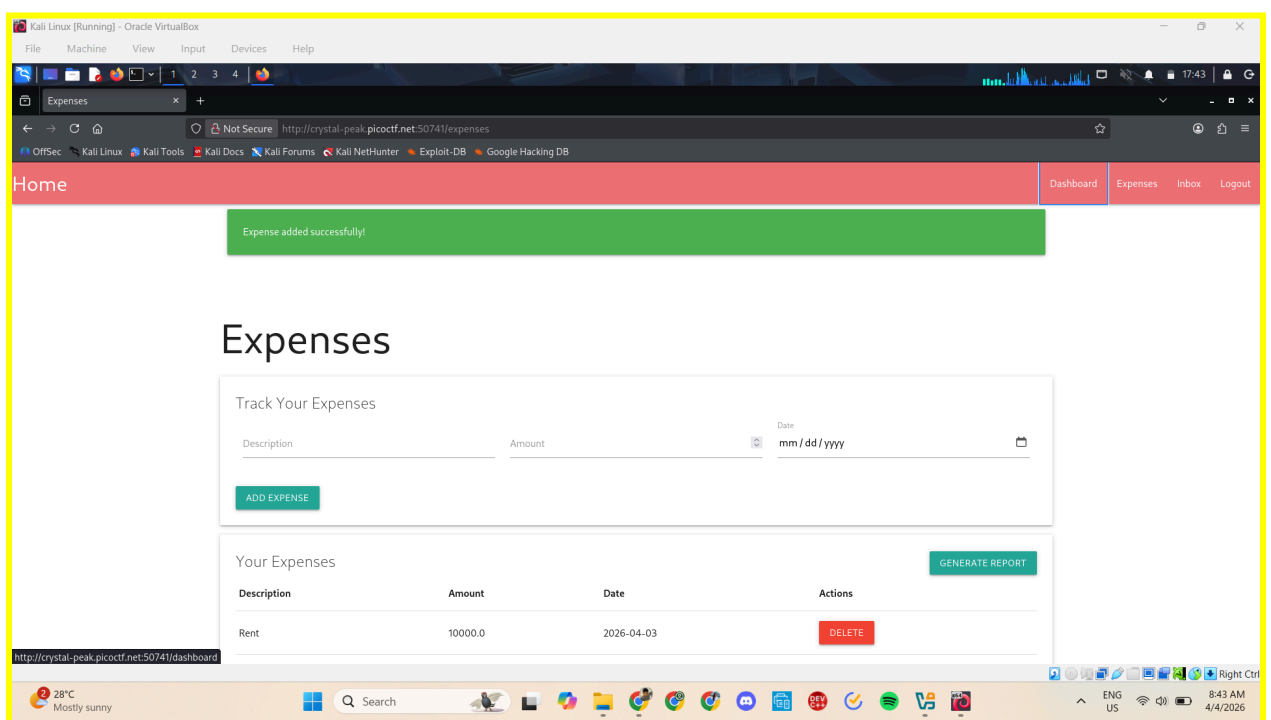
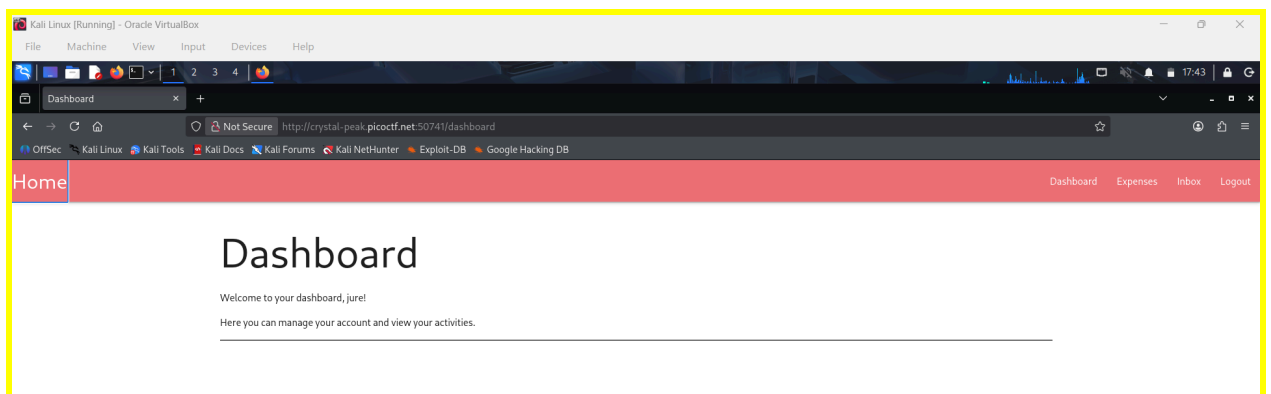


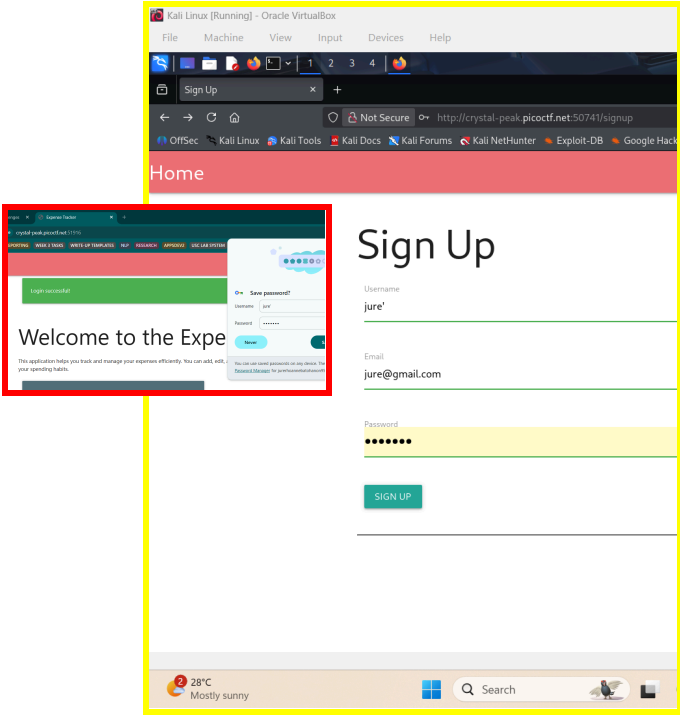
First, a new user account was created through the sign-up page, and the account was then used to log in successfully. After logging in, the user is directed to the Home page, which displays a welcome message, a brief description of the application, and navigation options in the upper-right corner such as Dashboard, Expenses, Inbox, and Logout.



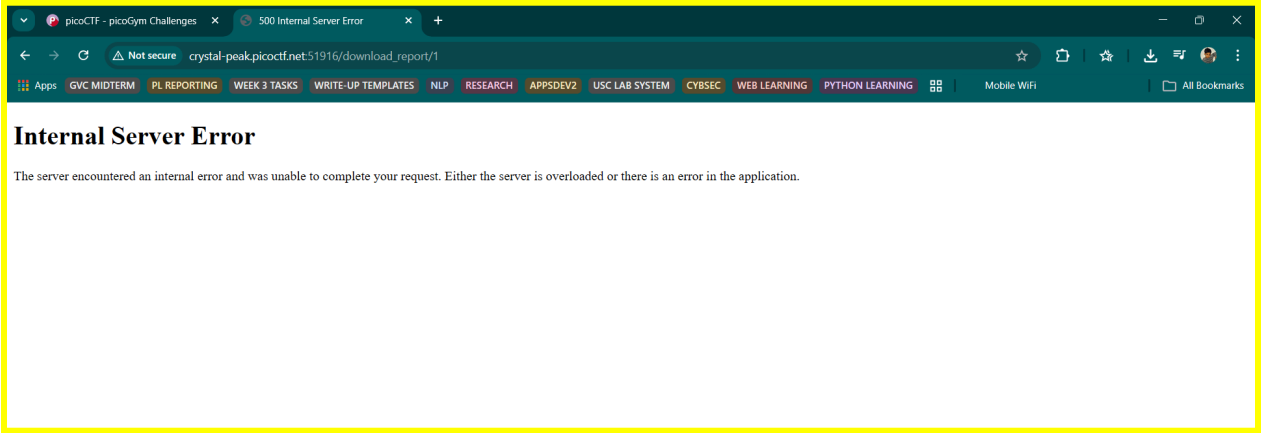
The **Dashboard** simply displays a welcome message with the username along with the text “Here you can manage your account and view activities,” but it does not provide any additional functionality and mainly serves as a static display. The **Expenses** section allows the user to add a new expense by entering the **description, amount, and date**, enabling them to manage and track their expense records. It also lists all added expenses and provides an option to generate a report.

The **Inbox** section displays the requested expense reports, showing details such as the **subject** (expense report with the corresponding date and time), the **report file** (in CSV format), the **date** it was generated, and the available **action** (download). To simulate normal user behavior, several expenses were added (e.g., rent and water) with their corresponding amounts and dates. After entering these expenses, the **Generate Report** feature was used, which produced a downloadable report file that was sent to the user's inbox. Opening the report confirmed that it contained the same expense data that had been entered.



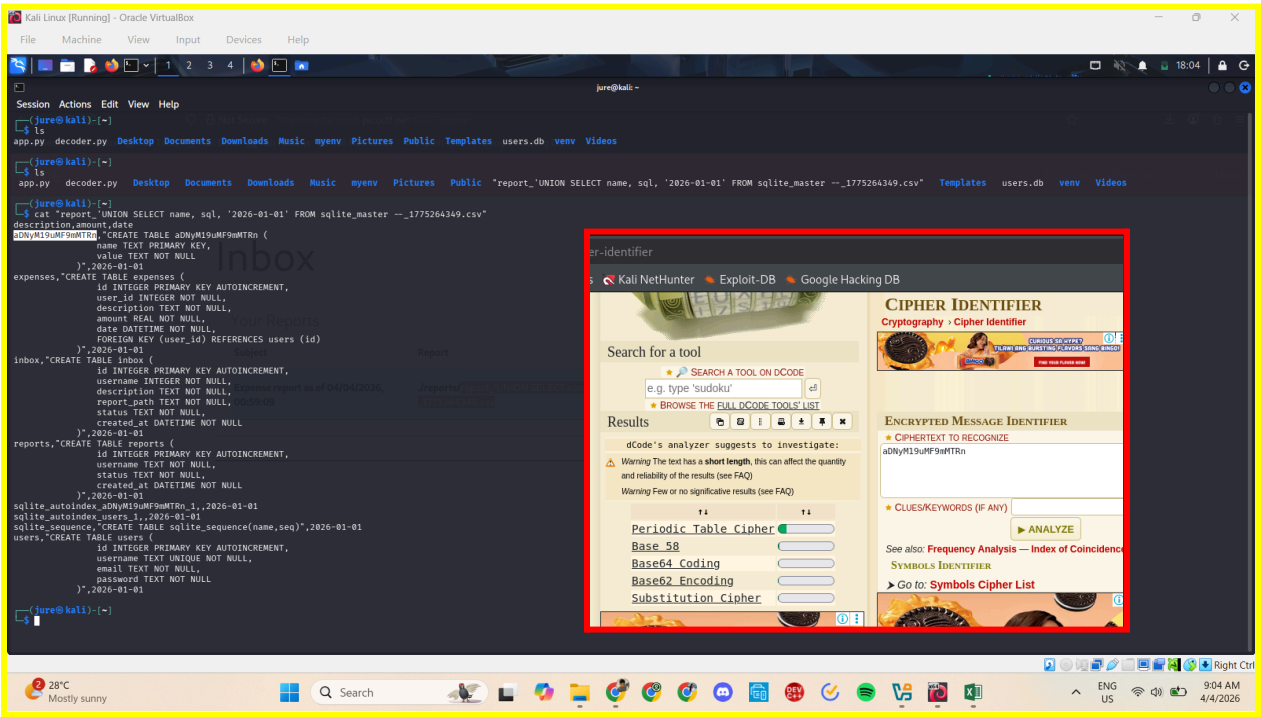
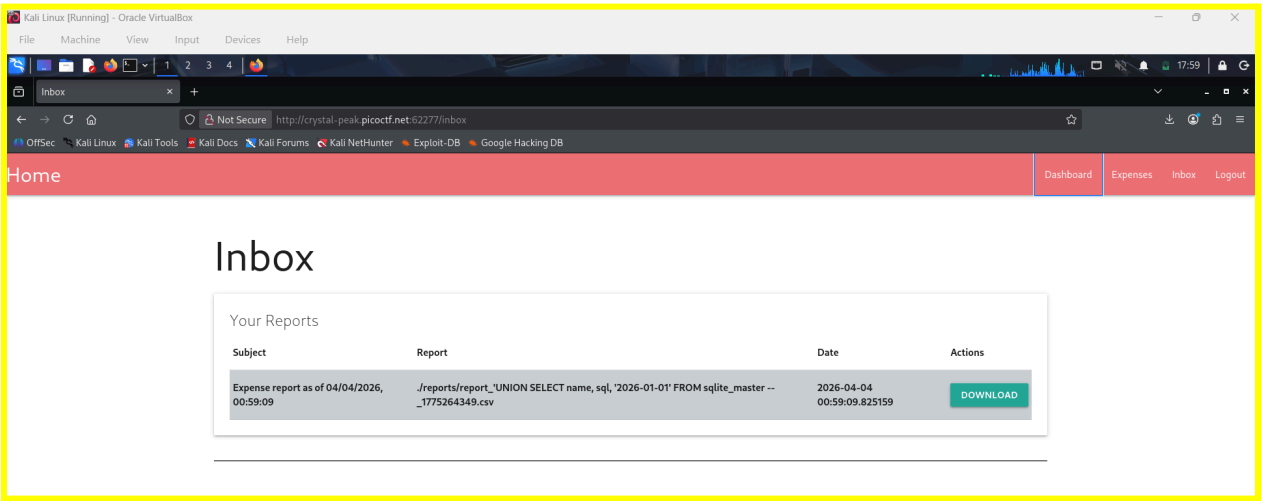


Next, the application was tested for **SQL injection vulnerabilities**. A simple test was performed by inserting an apostrophe (') into the username during registration. Surprisingly, the application allowed the registration even with the apostrophe in the username. After logging in with this account and generating a report again, the report generation process failed with an “unrecognized token” error. This indicated that the apostrophe had broken the SQL query being executed by the server. From this observation, it was concluded that the username field was vulnerable to SQL injection. However, the vulnerability did not occur immediately during login. Instead, the injected input was stored during registration and later executed when generating the report. This behavior identified the vulnerability as a Second-Order SQL Injection, where malicious input is stored in the database first and executed later by another process.

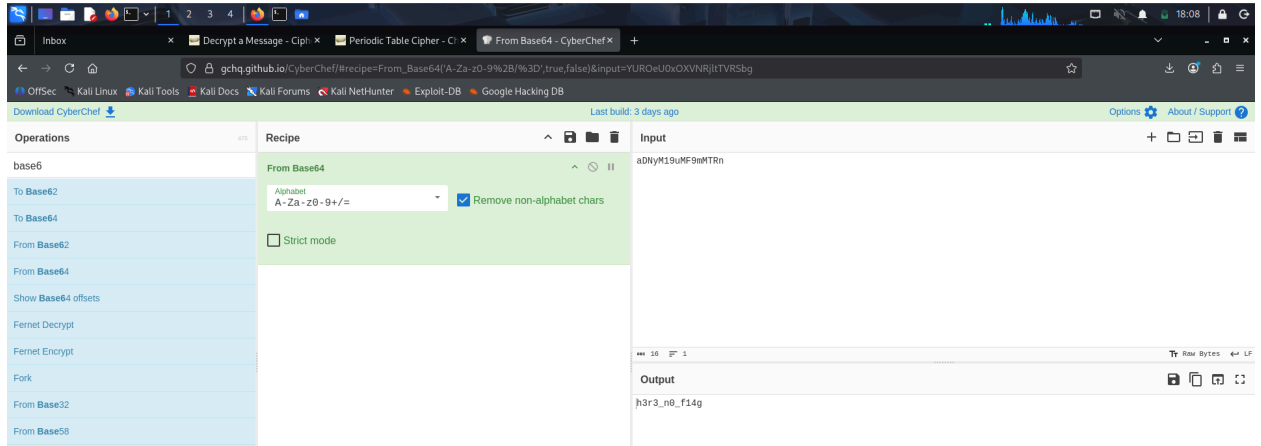


After confirming the vulnerability, the next step was to extract information from the database. A **UNION SELECT SQL injection payload** was inserted into the username field during registration. Specifically, I used ***‘UNION SELECT name, sql, ‘2026-01-01’ FROM sqlite_master --***.The payload attempted to query the database schema by selecting information from the **sqlite_master** table, which stores metadata about tables and database structures in SQLite. Once the malicious username was registered and used to log in, a new expense was added, and a report was generated again. The report file was then downloaded and inspected using the **cat** command. Instead of normal report data, the output contained

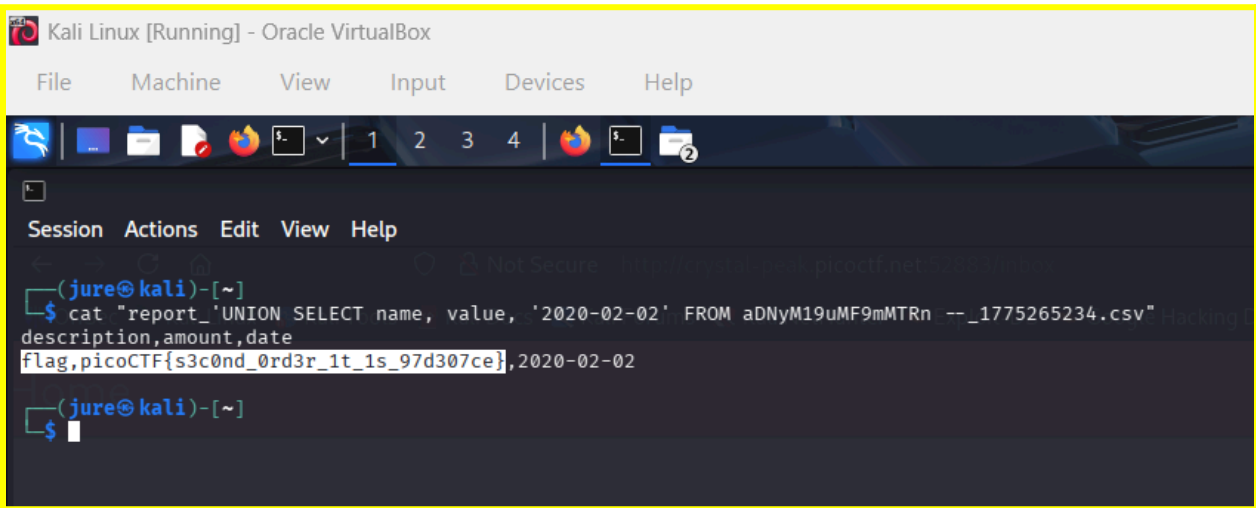
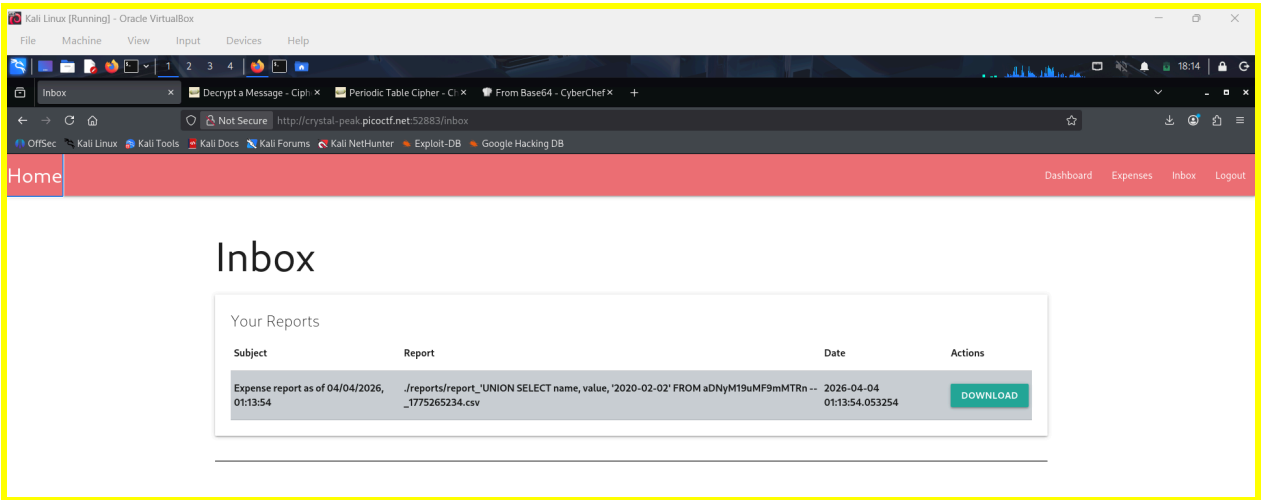
database schema information, including table definitions and encoded strings. This confirmed that the SQL injection was successfully leaking database information.



Among the extracted data, a suspicious encoded string was identified (the one highlighted above). The string was first analyzed using an online cipher identifier, which suggested several possible encoding methods. After testing the recommended encodings, it was determined that the string was Base64 encoded, so it was decoded to examine its content. Although the decoding revealed some information, it indicated that the flag might be stored inside a specific table in the database.



To retrieve the actual contents of that table, another SQL injection payload was crafted. This payload used UNION SELECT again, but this time it selected specific fields (name, value, and a placeholder date) from the suspicious table identified earlier. A new user was registered using this payload as the username, then logged in again. After logging in, an expense was added and the Generate Report feature was used once more. The downloaded report was again inspected using the terminal. This time, the output revealed the contents of the targeted table, which included the challenge flag.



ORDER ORDER

Hard

Web Exploitation

picoCTF 2026

AUTHOR: DARKRAICG492

Description

Can you try to get the flag from our website. I've prepared my queries everywhere! I think!

You can get started [here](#) to find the flag!

This challenge launches an instance on demand.

Its current status is: **RUNNING**

Instance Time Remaining: **14:52**

Restart Instance

Hints ?

1

What does order in SQL Injection mean?

693 users solved

91% Liked

picoCTF{s3c0nd_0rd3r_1t_1s_97d307ce}

Submit Flag

Hurray! You solved this challenge.

Basically, the solution involved identifying a **second-order SQL injection vulnerability**, leaking the database schema, identifying the relevant table, and finally extracting the table contents where the flag was stored.

Reflection:

This challenge helped me understand how a seemingly normal web application can still contain hidden vulnerabilities despite having basic features like user accounts, expense tracking, and report generation. By interacting with the system as a regular user and observing how data moves between the Expenses, Report Generation, and Inbox features, I realized the importance of carefully analyzing application functionality to identify potential security weaknesses. Overall, the activity emphasized how understanding normal system behavior is essential before attempting to discover and exploit vulnerabilities.